ResearchPilot

技术文档

系统设计、AI 技术方案与实现说明

项目概况

项目名称	ResearchPilot 面向科研阅读与写作的本地优先 AI 助手
文档类型	技术文档（项目交付）
版本	v0.5.0
代码仓库	https://github.com/Senven1006/ResearchPilot
技术栈	FastAPI · ChromaDB · PyMuPDF · RapidOCR · React 19 · Vite

v0.5.0 · 2026-10

仓库：https://github.com/Senven1006/ResearchPilot

目录

1. 项目概述 4

1.1 项目背景 4

1.2 建设目标 4

1.3 交付范围 4

1.4 关键设计取舍 5

2. 需求分析 6

2.1 用户与场景 6

2.2 功能需求 6

2.3 非功能需求 6

2.4 约束与假设 7

3. 系统架构设计 8

3.1 总体架构 8

3.2 模块职责 8

3.3 核心数据流 9

3.4 存储设计 9

3.5 接口清单（26 个） 9

3.6 部署拓扑 10

4. AI 技术方案 11

4.1 任务与选型 11

4.2 集成方式 11

4.3 检索优化策略 11

4.4 生成与核验优化 12

4.5 性能与成本优化 12

4.6 风险与对策 12

5. 核心功能实现 ····· 14
5.1 文件管理与两级去重 ····· 14
5.2 版式感知的结构化解析 ····· 14
5.3 抗幻觉问答 ····· 14
5.4 多文对比推理 ····· 14
5.5 版式阅读与双向跳转 ····· 15
5.6 翻译与中英对照 ····· 15
5.7 写作 Copilot ····· 15
5.8 数据可视化与图注 ····· 15
6. 测试与部署 ····· 16
6.1 测试环境 ····· 16
6.2 测试方法 ····· 16
6.3 测试结果 ····· 16
6.4 代码质量与工程规范 ····· 17
6.5 部署步骤 ····· 18
6.6 已知限制 ····· 18
7. 创新点 ····· 20
7.1 技术创新点 ····· 20
7.2 应用创新点 ····· 20
7.3 工程实践创新点 ····· 20
7.4 创新点与需求、证据的对应关系 ····· 21
8. 总结与展望 ····· 22
8.1 总结 ····· 22
8.2 展望 ····· 22

1. 项目概述

1.1 项目背景

科研人员日常面对的核心工作可以概括成三件事：把论文读懂、把结论核实、把想法写成论文。现有工具把这三件事拆散在不同软件里——PDF 阅读器负责看、笔记软件负责记、通用大模型负责"聊"，而三者的上下文互不相通。由此产生三个具体痛点：

- 结构与证据丢失：PDF 是排版格式而非语义格式，复制出来的正文会丢掉标题层级、表格行列、图表编号与公式，导致"看过"却"查不到"。
- 幻觉代价高：通用大模型可以流畅地编造实验设置、作者机构与参考文献；科研场景下这类错误无法容忍。
- 链路断裂：检索到的证据无法回溯到原文位置；阅读时的疑问无法直接变成写作素材；写作时的参考文献无法确认真实存在。

ResearchPilot 的定位是：把一篇（或一批）**PDF** 变成可检索、可核对、可对照阅读、可复用的结构化知识，并围绕它提供问答、对比、翻译与写作四类能力。

1.2 建设目标

目标	可度量的验收口径
结构化理解	单篇 15 页论文 2 秒内产出标题树、表格、图表、公式、参考文献与附录结构
抗幻觉问答	每条结论绑定"论文 + 页码 + 章节"锚点；断言级核验给出已核验/弱支持/未支持三态
多文推理	跨论文对比时每篇保底 2 条证据，并标注"仅一侧有据"
版式阅读	标题树导航、表格/图表/公式分栏呈现、正文引用与文献双向跳转
翻译	划词翻译 2 秒级返回；按章节渐进翻译，读完一节再译下一节
写作辅助	由 idea 生成三段框架，参考文献只来自库内真实论文，越界引用可检出
可运行性	无 GPU、无本地模型环境（仅在线模型可用）也能跑通全流程

1.3 交付范围

交付物	内容
后端服务	FastAPI 应用，8 个 Python 模块、5390 行代码（非空行）、26 个 HTTP 接口
前端应用	React 19 + Vite 单页应用，3 个界面（问答 / 结构化阅读 / 写作 Copilot），2992 行代码
数据层	ChromaDB 持久化向量库 + 本地文件缓存（原件留档、解析结果、图表裁图、译文、图表 SVG）
质量保障	离线 pytest 套件（16 个用例）+ 前端查重回归（9 条断言）+ GitHub Actions 双流水线（后端测试 / 前端静态检查、查重回归与构建）
工程文档	README、环境变量模板、依赖清单、本技术文档

1.4 关键设计取舍

> 第一原则：**能给出出处就说，给不出就明确说不知道**。所有结论都绑定到可核查的证据锚点上，宁可弃答也不编造。

> 第二原则：**本地优先、渐进降级**。默认走本地模型，缺 Ollama 时自动改用在线模型与离线哈希嵌入，功能不中断，只在界面上标注当前后端与质量差异。

2. 需求分析

2.1 用户与场景

角色	典型场景	关键诉求
研究生 / 科研人员	精读一篇论文并追问问法、超参数、实验设置	答案必须能被原文核对
综述撰写者	对比多篇论文的技术路线与指标演化	双侧都有证据，缺失要明确标出
非母语写作者	阅读英文论文并把自己的中文摘要译成学术英文	术语与公式不能被"翻译掉"
论文作者	从一个 idea 起稿框架、需要真实参考文献与架构图	参考文献不能编造

2.2 功能需求

编号	需求	验收要点
F1	单篇 / 批量 PDF 上传与 CRUD	单次最多 20 篇、单篇 30 MB / 600 页；失败文件不影响整批
F2	内容级与语义级去重	字节相同直接提示；不同版本按正文相似度给出保留建议
F3	细粒度结构解析	标题/作者/机构/摘要、一至三级标题树、表格、图表、公式、参考文献、附录
F4	扫描件 OCR	无文本层页面自动渲染并识别，标注已 OCR 页数与上限
F5	抗幻觉问答	证据锚点、断言核验、证据不足弃答
F6	多文对比推理	按论文配额取证、输出对照表、标注一侧缺失
F7	版式阅读与双向跳转	标题树导航；正文 [n] 与文献条目互相跳转
F8	划词翻译与整篇翻译	划词 2 秒级；按章节渐进；公式/文献保持原文
F9	中英双栏对照	双栏独立滚动且位置联动
F10	论文框架生成	Abstract / Introduction / Related Work；参考文献仅取自库内
F11	拓扑图脚本生成	Mermaid / Graphviz / TikZ / Matplotlib 四份并做静态校验
F12	CSV 数据可视化	画像、统计表、折线/柱状/三维雷达图、学术图注
F13	中→英学术翻译	学术语域，公式与指标名保留英文

2.3 非功能需求

类别	要求	当前实现
性能	单篇 15 页解析 ≤ 3 秒	实测 1.3-1.8 秒（纯文本层）
性能	划词翻译 ≤ 3 秒	实测 2.0 秒（DeepSeek），命中缓存 0 秒
可用性	无本地模型也能运行	嵌入降级为离线哈希、对话回落在线模型
可追溯	任何结论可定位到页码与章节	证据单元带 page / section / anchor_type / ref
数据安全	API Key 不入库、数据留在本地	.env 已 gitignore；向量与缓存均在本地 data/
可维护	模块职责单一、注释覆盖关键算法	8 个后端模块按职责拆分，关键函数均有中文文档字符串
成本	翻译与问答可控	章节级翻译 + 段落哈希缓存，二次打开零模型调用

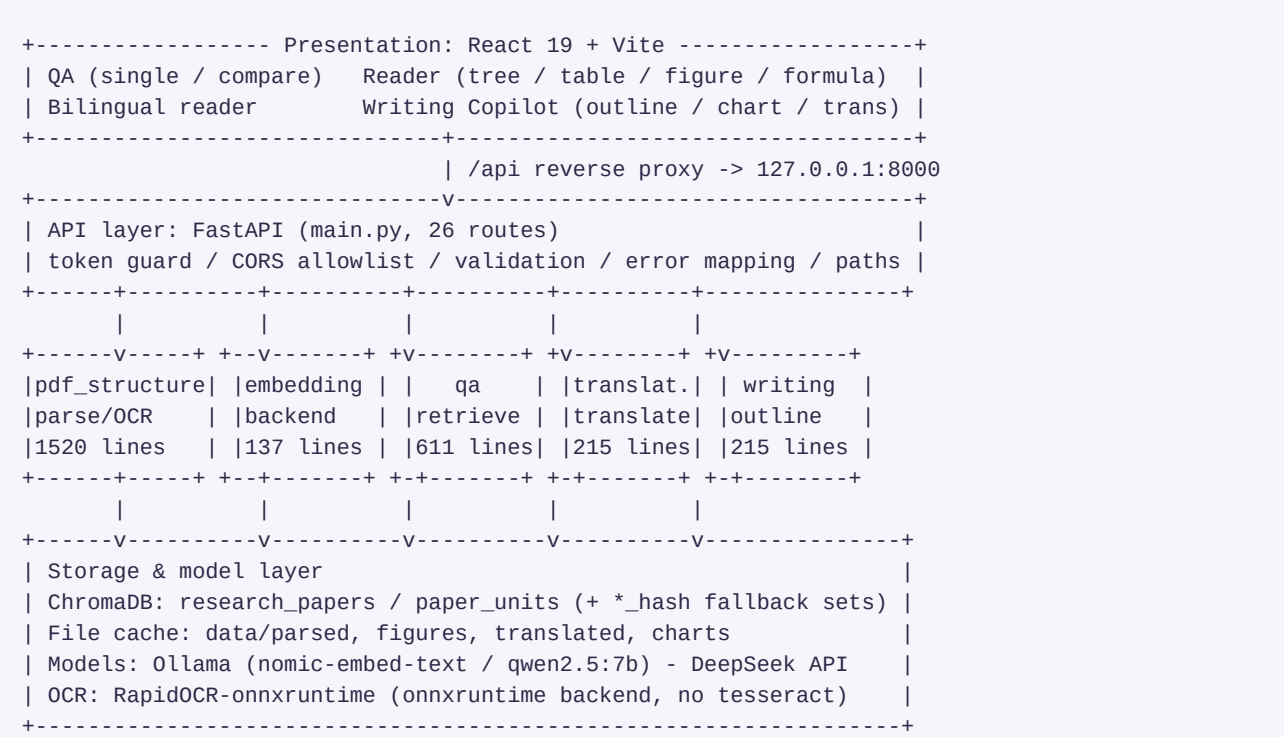
2.4 约束与假设

- 单机部署，无 GPU 假设成立；模型能力依赖 Ollama 或在线 API。
- 扫描件只做文字级 OCR，不还原表格与公式结构（版面分析需要额外模型）。
- 公式为近似 LaTeX（字体符号映射 + 上下标推断），不追求像素级排版还原。
- 无外部文献检索能力，因此"无幻觉参考文献"以"只允许引用库内论文"来保证。

3. 系统架构设计

3.1 总体架构

系统采用四层结构，自上而下依次是展示层、接口层、能力层、存储与模型层。层间只通过明确的函数签名或 HTTP 契约交互，能力层不直接依赖具体模型（嵌入函数以参数注入），因此可以整体替换模型而不改动业务代码。



上图中各层职责与模块行数对应第 3.2 节；`_hash` 后缀是嵌入降级时使用的独立集合。

3.2 模块职责

模块	职责	关键导出
main.py	路由装配、参数校验、集合访问、模型路由	ask_paper / index_one_pdf / hybrid_retrieve / resolve_model
pdf_structure.py	版式感知解析、OCR、标题树、表格/图表/公式/文献/附录	StructureParser
embedding.py	嵌入后端探测与降级、哈希向量	detect / hash_embed / collection_suffix
qa.py	检索单元构建、BM25、RRF、证据配额、断言核验	build_units / verify_claims
semantic_dedup.py	句子级语义对齐、版本建议	compare_documents / build_recommendation
translation.py	分章节翻译、批量并发、磁盘缓存	translate_structure / translate_texts
writing.py	无幻觉框架、拓扑图脚本、学术翻译	build_outline_prompt / validate_citations
dataviz.py	CSV 画像、SVG 图表、确定性图注	build_report

3.3 核心数据流

```
PDF -> page spans (font, size, bbox)
    -> scanned-page check (chars < 40 and image ratio > 50%) -> 200dpi render -> RapidOCR
    -> structure: TOC or font clustering -> heading tree; find_tables -> tables;
                  caption-based figure crop; math-font lines -> approx LaTeX;
                  references and appendix split into standalone sections
    -> dual write: 800-char paragraph chunks -> research_papers
                  anchored units (paragraph/table/figure/equation/reference/meta) ->
 paper_units
```

```
question -> hard terms (numbers / model ids / greek letters) + intent
 (table/formula/reference/meta)
        -> vector recall 40 + BM25 recall 40 -> RRF fusion
        -> re-weight: entity coverage, literal number hit, intent anchor,
           section-title overlap, reference penalty
        -> quota selection (compare mode: at least 2 per paper)
        -> evidence with anchors (paper / page / section / table or equation id)
        -> LLM answer -> claim split -> semantic support x number consistency -> 3-state
 verdict
```

3.4 存储设计

存储	内容	关键字段
research_papers	段落块（800 字/块，重叠 120）	filename、file_sha256、page、chunk_number、arxiv_id、version
paper_units	结构化检索单元	unit_id、anchor_type、anchor_label、section、section_level、page、ref、year、title
*_hash	降级模式的两套同名集合	同上，维度 384（Ollama 为 768）
data/parsed/*.json	完整解析结果（含 Markdown 版本）	meta、outline、sections、tables、figures、equations、references、appendix、stats、limitations
data/papers/<sha256>.pdf	入库时的原件留档，供 GET /papers/file/{sha256} 取回	以内容哈希命名，天然去重；缺失不影响索引
data/figures/<sha>/	图表裁图 PNG	按页与序号命名
data/translated/<sha>.<model>.json	段落级译文缓存	段落哈希 → 译文
data/charts/<stem>/	可视化 SVG	折线/柱状/雷达

集合按嵌入后端隔离（`_hash` 后缀）是关键决策：不同后端的向量维度与语义空间都不兼容，混写会让检索彻底失真，而隔离后切换后端不需要重建另一侧的索引。

3.5 接口清单（26 个）

分组	路径	说明
基础	GET /、GET /health、GET /papers/status	健康检查、嵌入后端与库统计，并标记原件可下载
索引	POST /papers/index、POST /papers/batch-index	单篇 / 批量入库，structure=true 同步建单元并留档原件
原件	GET /papers/file/{sha256}	取回留档 PDF；sha256 需为 64 位十六进制，缺失返回 404
解析	POST /papers/parse、POST /papers/structure、GET /papers/structure/{id}、GET /papers/structure/{id}/figures/{name}	文本抽取、结构化解析、结果缓存、裁图
去重	POST /papers/duplicates/check-file、GET /papers/duplicates/check、POST /papers/duplicates/semantic-check	字节去重、版本识别、正文语义比对
问答	POST /papers/ask	mode=single|compare，返回证据/核验/覆盖率
翻译	POST /translate、POST /papers/structure/{id}/translate	划词、分章节（可选 sections）
写作	POST /writing/outline、POST /writing/diagrams、POST /writing/translate	框架、拓扑图脚本、学术英文
可视化	POST /dataviz/report、GET /dataviz/charts/{stem}/{name}	CSV 报告与 SVG 图表

3.6 部署拓扑

单机部署，三个进程：后端 `uvicorn main:app --port 8000`、前端 `vite`（开发 5173 / 生产静态托管）、Ollama `11434`（可选）。前端通过 `/api` 前缀代理到后端，避免跨域配置；数据全部落在仓库同级的 `data/` 目录，可直接打包迁移。

对外暴露时有两道可选加固，默认关闭以保持本机开发零配置：

- 令牌守卫：一个 HTTP 中间件拦截除 `/`、`/health`、`/docs`、`/redoc`、`/openapi.json` 之外的请求，要求 `X-API-Token` 头或 `?token=` 参数，用 `secrets.compare_digest` 恒时比较。`OPTIONS` 预检必须放行——浏览器预检不会携带自定义头，否则开启令牌后跨域请求会在预检阶段被 401 打回（该问题由真实服务器冒烟测试发现，已补用例）。前端在 `src/api.js` 里统一注入请求头，`<img src>` 类请求（裁图、SVG 图表）走查询参数。
- **CORS** 白名单：`ALLOW_ORIGINS` 逗号分隔，默认只放行 5173。

这两项解决的是"本机/内网自用"场景，不构成多租户隔离——token 会进入浏览器 bundle 与 URL，公网部署仍需 HTTPS 反代与账号体系。

4. AI 技术方案

4.1 任务与选型

子任务	选用方案	选型理由	放弃的备选
结构抽取	PyMuPDF span 级 API	单库同时提供字号/字体/坐标、get_toc、find_tables、图像与绘制信息，无需多库拼接	pdfplumber（无 OCR）、pypdf（无版面）、Marker（依赖 torch 与模型下载）
扫描件 OCR	RapidOCR-onnxruntime	纯 pip 安装、自带 PP-OCR 模型、跑在 onnxruntime 上，不依赖系统 tesseract	Tesseract（需外部二进制）、PaddleOCR（依赖重）、云 OCR（隐私与费用）
语义嵌入	nomic-embed-text（Ollama）	本地推理、768 维、中英混排表现稳定	OpenAI embedding（需外网与费用）、句向量模型（需 torch）
降级嵌入	自研哈希向量（384 维）	零依赖、确定性、可离线；保证无 Ollama 时全链路可用	无（原先的方案是直接不可用）
对话生成	DeepSeek（在线）/ qwen2.5:7b（本地）	在线模型质量高、成本低；本地模型可离线；两者通过统一接口切换	单一模型（任一不可用即中断）
向量库	ChromaDB 持久化	单文件部署、自带元数据过滤、支持 upsert 与集合隔离	FAISS（无元数据过滤）、Milvus（需服务化）
图表绘制	自研 SVG 生成	环境无法安装 matplotlib，SVG 无损放大、无第三方依赖、可文本检索	matplotlib（pip 源不可得）、plotly（需前端运行）
翻译	分章节 + 批量并发 + 缓存	输出 token 是瓶颈，按章节切分把等待时间降到可读级别	整篇一次性翻译（等待数分钟）

4.2 集成方式

```
# resolve_model 的决策顺序
await embedding.detect()            # cached /api/tags probe (60s TTL)
if request == "local" and local_ready:      return "local"
if request == "local" and not local_ready:  return "deepseek" if key else 503
if request == "auto":                       return "deepseek" if key else ("local" if
 local_ready else 503)
```

4.3 检索优化策略

策略	具体做法	实测效果
结构化单元替代定长切块	表格整表成块、文献逐条成块、段落按句边界切分	"超参数 ·1/·从召不回变为第 1 位命中
单复数与中英术语映射	GPUs→gpu；中文技术词映射英文（注意力→attention、显卡→GPU 等 40 余条）	中文提问可命中英文正文
数字逐字命中加权	命中数字/型号 +0.35（归一化）	"28.4 BLEU""8 张 P100"稳定命中
实体覆盖率	点名实体一个都没出现时分数 ×0.55	避免"表2 概述"类泛化段落挤掉硬件段落
意图锚点	问表格/公式/文献时对应类型 ×1.12 问正文时文献 ×0.7	文献条目不再抢占正文证据位
章节标题重合	问题词与章节标题词重合时最多 ×1.2	"注意力头"命中 3.2.2 节
实体强制入候选	含点名实体的单元强制进入候选集	防止被向量排名过滤掉
RRF 融合	向量与 BM25 按名次融合（k=60）	兼顾语义与字面

4.4 生成与核验优化

- 断言级核验：把答案切句，逐句取所引证据，计算语义支持度（余弦）与数字一致性（数字必须出现在所引证据或其页码/章节锚点中），合成置信度并映射为三态。实测在 v1 论文上 GPU 类问题支持度 87.5%，伪造数字（"16 张 V100"）被判未支持。
- 降级模式只做能做的：无语义嵌入时，跨语言语义相似度必然偏低，若沿用三态会把正确回答误判为编造。因此降级时只校验数字与引用编号，结论标"未核验"并说明原因。
- 失败不丢证据：模型异常时接口仍返回 200 与完整证据，前端可继续核对原文。

4.5 性能与成本优化

优化	做法	数据
OCR 提速	关闭方向分类器（use_cls=False），引擎进程级复用	单页 11.7s → 3.7s
解析提速	只对扫描页 OCR；表格按页并集扫描	15 页 1.3·1.6s；120 页 11.1s
翻译吞吐	批次 5 段 + 并发 4 + 失败 3 次退避重试	95 段从 101s 降至约 40s 量级
翻译等待感	按章节请求，读完一节再点"继续翻译"	单节 13·27 可读，而非整篇数分钟
缓存	段落哈希缓存（按文档 + 模型隔离）	二次打开 0 次模型调用
成本	max_blocks 上限、参考文献不翻译	整篇翻译 token 量下降约 35%
降级可用	嵌入哈希 < 10ms/批（本地计算）	无 Ollama 时索引 1.9s、问答 3.3s

4.6 风险与对策

风险	影响	对策
本地模型缺失 / Ollama 未启动	原设计下全链路不可用	嵌入降级 + 对话回落 + 界面标注
在线 API 限流或抖动	单次请求失败	3 次退避重试；失败仍返回证据；可切本地模型
表格抽取错行	表格数据不准	题注配对 + 超宽/稀疏过滤；界面标注置信度与限制
翻译等待过长	体验差	章节渐进 + 缓存 + 并发
版本空间混写	检索失真	集合按后端隔离（_hash 后缀）
参考文献造假	学术风险	只允许引用库内论文，越界编号检出

5. 核心功能实现

5.1 文件管理与两级去重

上传入口支持单篇与批量（上限 20 篇），每篇先做三重校验：扩展名、`%PDF-` 魔数、体积（30 MB）。批量接口逐文件处理，单文件失败只记入 `results[].error`，不影响其余文件。

去重分两级：

- 字节级：`sha256(content)` 写入元数据，命中即提示"内容完全相同的已有论文"。
- 语义级：同名 arXiv 编号的不同版本（如 v1 与 v7）用正文比对。把两版正文切句、去重，先按规范化字符串精确配对，剩余句子再用嵌入做一对一贪心匹配（阈值 0.88），得到 matched / added / removed，相似度取 `2M/(N1+N2)`；再结合版本号先后给出保留建议。

实测（v7 与已入库 v1 对比）：相似度 90.0%，匹配 297 句（精确 198 + 语义 99），新增 46 句、删除 20 句，变化集中在第 1·1页，最终建议为"保留 v7，并将 v1 标记为旧版本"（`relation=incoming_newer`）。

两级去重都只查"已入库"的数据，因此存在一个盲区：库里为空时同时选中 v1 与 v7，两份都还没入库，两次检查都查不到对方，用户不会看到任何提示。补的做法是在前端上传前先做一次批次内比对（`src/precheck.js`）：按文件名里的 arXiv 编号分组，同一编号出现多个版本就提示"建议只上传最新版本"，编号缺失或版本号相同则提示人工核对。这一层只看文件名、成本为零，但能覆盖最高频的那次操作；没有编号的文件改名后仍查不到，需要靠入库后的语义查重兜底。

关键实现细节：`load_indexed_pages()` 从 800 字重叠分块精确还原每页正文——相邻分块只在开头重叠 `CHUNK_OVERLAP=120` 字符，按"最长重叠后缀"拼接即可，不需要额外存原文快照。

原件留档的入口在侧栏论文列表：每篇论文右侧按 `files[].downloadable` 渲染一个下载图标，链接是 `<a href="/api/papers/file/<sha256>?token=…">`。之所以用链接而不是 `fetch`，是因为浏览器原生下载不需要把整份 PDF 读进内存；代价是资源类请求无法带自定义头，因此令牌走查询参数（与裁图、SVG 图表同一处理方式）。

5.2 版式感知的结构化解析

5.3 抗幻觉问答

5.4 多文对比推理

对比模式（`mode=compare`）对每篇论文设置配额 `max(2, top_k // 篇数)`，先按配额填充再按总分补足，保证两侧都有证据。提示词要求输出"维度 ×论文"的 Markdown 对照表，每格标注证据编号，单侧缺失时写"仅一侧有据/无证据"。响应额外给出 `coverage`（每篇证据条数），前端据此显示证据分布。

5.5 版式阅读与双向跳转

结构化面板把解析结果分五栏呈现：标题树与正文、表格、图表、公式、参考文献。正文中的 `[12]`、`[3, 4]`、`[12·14]` 会被渲染成可点击标签：点击跳到参考文献并高亮该条；文献条目反向显示"被引 N 次 · 跳回：第 X 页"，点击回到第一个引用它的正文块。反向索引在前端由正文标记实时构建（`Map<编号, 引用位置[]>`），不额外占存储。

5.6 翻译与中英对照

5.7 写作 Copilot

5.8 数据可视化与图注

`dataviz.build_report` 依次完成：解析 CSV（自动识别 UTF-8/GBK 等编码）→ 列画像（数值/类别/文本，含缺失、唯一值、min/max/mean/median/std）→ 统计表（Markdown + HTML）→ 图表 → 图注。

图表用纯 Python 生成 SVG：折线（含面积填充与网格）、柱状（含数值标签）、三维雷达（斜投影分层 + 半透明多边形）。图注由统计量确定性生成：折线给出首末值与相对变化、最陡区间、峰值位置；柱状给出最高/最低与极差占均值比例；雷达给出各指标归一化最大值及解读提示。这样做的好处是图注不可能出现"说上升但数据下降"的幻觉。

6. 测试与部署

6.1 测试环境

项	配置
操作系统	Windows 11，无 NVIDIA GPU
Python	3.12.7（backend/.venv）
关键依赖	fastapi 0.143、chromadb 1.5.9、pymupdf 1.28.2、rapidocr-onnxruntime 1.4.4、numpy 2.5.4
前端	Node.js 24、React 19.2、Vite 8.3
模型	DeepSeek deepseek-flash（在线）；Ollama qwen2.5:7b + nomic-embed-text（本地）
测试语料	arXiv 1706.03762 的 v1 与 v7（15 页论文），合成的 120 页"书籍"，合成的扫描件（页面渲染为图片后重建 PDF）

6.2 测试方法

- 真实语料回归：用 v1/v7 两版论文覆盖解析、去重、检索、翻译、对比全链路。
- 离线自动化回归：`backend/tests/` 的 16 个用例（15 条接口级 + 1 条环境约束），用 `CHROMA_PATH` 指向临时目录、`OLLAMA_BASE_URL` 指向关闭端口强制走降级嵌入、`DEEPSEEK_API_KEY` 清空且用桩函数替换 `generate_answer`/`resolve_model`，因此不联网、不依赖 Ollama 与真实 Key；PDF 语料由 PyMuPDF 现场合成（标题页、三级标题、含"8 NVIDIA P100 GPUs"的表格、参考文献条目）。
- 降级验证：把 `OLLAMA_BASE_URL` 指向关闭端口模拟"未启动 Ollama"；另用本地 mock 服务（实现 `/api/tags`、`/api/embed`、`/api/chat`）验证 Ollama 可用分支，避免依赖真实服务。
- 对抗验证：人为构造含错误数字的答案，检查核验是否判为"未支持"；构造未授权引用编号，检查是否被检出。
- 边界验证：615 页文档应被拒绝；空库生成框架应拒绝；损坏 PDF 应返回 400。

6.3 测试结果

模块	用例	结果
批量上传	2 篇正常 PDF	200，两篇均 indexed，单元 243 / 252
批量上传	混合非法文件（空文本 PDF + .txt）	200，逐条给出失败原因，不中断
批量上传	21 个文件	413，提示上限 20
去重	v1/v7 语义比对	相似度 90.0%，建议保留 v7
去重	字节相同文件	命中并提示覆盖风险
解析	v1（15 页）	1.5s：标题树 22 节、表格 2、图 5、公式 23、文献 36
解析	120 页合成书籍	11.1s，doc_type=book，96 章节、16 表、40 图、184 公式

模块	用例	结果
解析	615 页	413，超上限拒绝
解析	扫描件（3 页）	OCR 3 页，标题按编号规则推出，标注"表格与公式未抽取"
问答	GPU 型号与数量	证据指向 5.2 节第 7 页，答案正确，语义核验 3/4 通过
问答	Adam ·1/·2/·	5.3 节原文第 1 位命中
问答	伪造数字（16 张 V100）	判为"未支持"，指出数字未出现
对比	v1 vs v7 训练差异	每篇 3 条证据，输出对照表并标"仅一侧有据"
翻译	划词	2.0s 返回，术语与公式保留
翻译	单章节	13·27s重复请求 0s（缓存）
翻译	整篇 217 段	首次 37 次调用，二次 0 次
降级	Ollama 未运行	嵌入 hash，索引 1.9s，问答 3.3s 走在线模型
写作	空库生成框架	400 + 明确原因（不编造）
写作	引用越界	[R2]、[R9] 被检出
可视化	10 行 × 5列 CSV	折线/雷达生成，图注含 -73.0% 与最陡区间
可视化	3 类 ×数值 CSV	柱状图生成，图注含最高/最低与极差占比
前端	构建与静态检查	vite build 339ms 通过；ESLint 0 error / 0 warning
自动化	python -m pytest（16 用例）	16 passed，3.2s，全程离线
自动化	原件留档与下载	/papers/status 的 files[] 两条均 downloadable=true；GET /papers/file/{sha256} 返回 2,125,019 字节、头为 %PDF-1.5，与留档文件大小一致
自动化	断言核验（对抗）	正确数字判"支持"，伪造数字被标出，降级模式降为 unverified
自动化	鉴权开关	无 token / 错 token → 401；header 与 ?token= 均 → 200；公开路径与 OPTIONS 预检放行
自动化	CI	GitHub Actions 后端 pytest + 前端 lint / 查重回归 / build 双流水线通过
自动化	批次内查重（npm run test:precheck）	9 条断言全过：v1+v7 同批、无版本号、同名重复、不同论文、编号嵌在长文件名、v2 与 v10 数值比较

6.4 代码质量与工程规范

- 后端 8 个模块均通过 AST 解析与导入检查；移除死代码（未调用的相似度函数、废弃的打分函数）与调试输出。
- 前端通过 ESLint 全量检查：修复未使用变量、Hooks 依赖缺失、正则共享状态副作用等问题，达到 0 error / 0 warning。
- 测试沉淀为可重复执行的套件：手工回归只做过一次就没人再跑，因此把它固化成 `backend/test`

s/（pytest 9.1.1，16 个用例）。约定是"每个后端能力至少有一条接口级用例，且用例必须离线可跑"——不联网、不要求 Ollama、不要求真实 Key，否则 CI 里必然被跳过。其中一条用例守的是环境约束而不是功能：两个 `requirements` 文件必须保持纯 ASCII，因为 pip 24.2 按系统 locale 解码需求文件，中文 Windows 的 GBK 遇到 UTF-8 中文注释会直接 `UnicodeDecodeError`（真机上踩过一次）。

- **CI** 守住回归线：[`.github/workflows/ci.yml`](.github/workflows/ci.yml) 两个 job，后端跑 `pip install -r requirements-dev.txt` + `pytest`，前端跑 `npm ci` + `npm run lint` + `npm run test:precheck` + `npm run build`，任何一步失败即红。前端那条查重回归是纯 Node 脚本（不依赖浏览器），因为"空库首次上传两个版本要不要提示"这种判断只发生在浏览器侧的批次内部，用接口测不到。
- 提交历史按"依赖与环境 → 解析 → 去重 → 检索核验 → 翻译 → 写作 → 图表 → 接口 → 前端 → 安全与留档 → 测试与 CI → 文档"的依赖顺序组织，每类改动单独成 Commit，便于回溯每一层能力的引入时间。

6.5 部署步骤

```
# 1) 后端
cd backend
python -m venv .venv && .venv/Scripts/activate
pip install -r requirements.txt
cp .env.example .env          # 填 DEEPSEEK_API_KEY 或启动 Ollama
uvicorn main:app --port 8000

# 2) 前端
cd ../frontend
npm install
npm run dev                   # http://127.0.0.1:5173

# 3) 提交前自检（可选，CI 会做同样的事）
cd ../backend && pip install -r requirements-dev.txt && python -m pytest
cd ../frontend && npm run lint && npm run test:precheck && npm run build
```

对外暴露前在 `backend/.env` 里设 `API_TOKEN`（并在 `frontend/.env` 里填同一个值到 `VITE_API_TOKEN`）、把 `ALLOW_ORIGINS` 收紧到实际前端地址。

生产部署建议：前端 `npm run build` 后由 Nginx 托管静态文件并把 `/api` 反代到 8000；后端用 `uvicorn --workers 2` 或 systemd 托管；`data/` 目录单独挂载并定期备份。

6.6 已知限制

限制	说明	影响面
扫描件的表格与公式	只做文字 OCR，不还原结构	扫描版论文的表格数据需人工核对
公式 LaTeX	近似重建，非像素级还原	复杂公式需对照原始文本
降级嵌入质量	哈希向量为词袋级，跨语言与同义改写的召回较弱	无 Ollama 时检索精度下降
翻译吞吐	受模型输出速度限制，全量翻译需数分钟	已用章节渐进缓解
拓扑图脚本	只做静态校验，不执行渲染	需用户在本地/在线渲染
页数上限	单文档 600 页	超长篇书籍需拆分
长文解析同步阻塞	解析在请求内完成，无进度与取消	120 页约 11s，更长文档体验下降
鉴权强度	单令牌 + CORS 白名单，token 会进 bundle 与 URL	适合单机/内网自用，非多租户方案
批次内查重只认文件名	同批次的多版本靠文件名里的 arXiv 编号识别	改名或没有编号的同论文两份不会被提前提示，入库后的语义查重仍能发现

7. 创新点

7.1 技术创新点

1. 结构化锚点检索（**Anchor-Aware RAG**）：不把整篇论文当无结构长文本，而是把表格、图表、公式、文献条目、元信息分别建成立检索单元并携带位置锚点。检索命中即得到"第几页、哪一节、哪张表"，把定位从"模型描述"变成"系统给定"。
2. 断言级抗幻觉核验：在生成之后再做一次可计算的检查——逐句取所引证据，计算语义支持度与数字一致性，输出三态结论。配合"引用编号白名单 + 越界正则校验"，把抗幻觉从提示词层面推进到可度量的工程层面。
3. 无 **Ollama** 环境下的自动降级：以可插拔嵌入后端（Ollama 语义嵌入 / 内置 384 维哈希嵌入）加集合隔离，实现"缺本地模型也能运行"；并在降级时自动切换核验模式，避免把正确回答误判为编造。
4. 分块精确还原原文：利用"定长切块 + 固定重叠"的可逆性，用"最长重叠后缀"拼接从向量库精确还原每页正文，使语义比对与原文核对不需要额外存储原文快照。
5. 无监督的版面结构推断：在无书签 PDF 上，用字号加权中位数、编号点分深度、标题文本定位与位置单调性约束组合推断标题树，并单独切分参考文献章节，解决了"章节吞并文末内容"的常见错误。
6. 确定性图注生成：图注由统计量（斜率、极差、最陡区间、峰值）直接生成，绕开大模型，从机制上排除"图注与数据不符"的幻觉。

7.2 应用创新点

1. 阅读—查证—写作闭环：同一份结构化解析结果同时服务于阅读导航、抗幻觉问答、跨文献对比、翻译与写作框架，避免在多个工具间反复搬运上下文。
2. 按章节渐进翻译的阅读节奏：把"等整篇翻译"改为"读一节、译一节"，把不可控的分钟级等待拆成可接受的十秒级等待，并提供"一次译完"作为备选。
3. 正文引用与文献条目的双向无损跳转：从 `[12]` 跳到文献、从文献跳回被引位置，形成可正反追溯的阅读路径，而非单向参考列表。
4. 对比模式的双侧证据保障：通过每篇保底配额与"仅一侧有据"标注，把跨文献对比从"模型自由发挥"约束为"有据可比、无据明说"。
5. 面向评审与教学的可解释性：界面同时展示证据锚点、核验结论、解析限制与当前模型后端，使用者能一眼判断"这条结论是怎么来的、可信到什么程度"。

7.3 工程实践创新点

1. 离线可测的 **AI** 系统：多数 RAG 项目"只能在有模型、有网络时验证"，本项目的测试套件通过"关端口强制降级 + 桩函数替换模型调用 + 现场合成 PDF"三件事，把整条 AI 链路变成可在 CI 里 3 秒跑完的确定性用例。

2. 降级路径本身被测试覆盖：降级不是兜底文档里的一句话，而是有用例断言的具体行为（后端标识 `hash`、集合带 `_hash` 后缀、核验模式自动切换、`grounding.score` 置空）。
3. 原件留档与内容哈希对齐：以 `sha256(内容)` 同时作为去重键、留档文件名与下载路由参数，三处共用同一标识，避免"索引说 A、文件是 B"的漂移。
4. 安全开关零成本：鉴权与 CORS 走环境变量，未配置时行为与本机开发完全一致，配置后即刻生效，不需要改代码分支。
5. 失败可见而非静默兜底：降级、留档失败、单元清理失败、译文命中缓存这类"没出事但状态变了"的情况一律在响应里给出布尔值或计数，界面据此显示，而不是悄悄替换结果。

7.4 创新点与需求、证据的对应关系

评审最常问"这条创新有没有落到可验证的地方"，对应关系如下表，证据位置均可在本文章节与代码中查证。

创新点	对应痛点	文中证据	验证方式
结构化锚点检索	证据回不到原文位置	3.4 存储设计、4.3 检索优化策略	证据卡片带页码与章节，越界引用被检出
断言级抗幻觉核验	回答敢不敢相信	4.4 生成与核验优化	伪造数字被判"未支持"，三态结论可复现
无 Ollama 自动降级	无 GPU / 无本地模型的可用性	4.2 集成方式、6.3 测试结果	关端口后仍能索引与作答，后端标识为 hash
章节渐进翻译	长文档阅读节奏与等待时长	5.6 翻译与中英对照	首次 37 次调用、二次 0 次（缓存命中）
原件留档与令牌守卫	交付级可用性与安全边界	3.4、3.6、6.3	downloadable 标记、401/200 四种鉴权用例
离线可测的 AI 链路	回归只能人工跑一次	6.2、6.3、6.4	16 个用例 3.2s 全程离线通过，CI 双流水线

上表只列可以复现的部分。扫描件的表格结构还原、公式的像素级重建、拓扑图的真实渲染都没有做到，已在 6.6 逐一列为已知限制，不计入创新点。

8. 总结与展望

8.1 总结

ResearchPilot 已完成从 PDF 到可核对知识的完整链路：结构化解析解决"看得懂结构"，锚点检索解决"找得到证据"，断言核验解决"敢不敢相信"，翻译与双栏对照解决"读得下去"，写作 Copilot 与数据可视化解决"用得上"。工程上以 5390 行后端与 2992 行前端实现 26 个接口与 3 个界面，在无 GPU、无本地模型的环境下仍可完整运行，并在真实论文上完成了端到端回归；同时以 16 个离线用例 + 9 条前端查重断言 + CI 双流水线把回归固化为可重复执行的流程。

过程中也暴露出若干真实问题并已修复：文末参考文献被并入最后一节、同页多章节的后续节为空、降级模式下核验误判、翻译等待过长、图表页无法滚动、开启令牌后 CORS 预检被拦、空库首次上传同一论文的两个版本时查重失灵（检查只查已入库数据，同批次两份互相看不见）。这些问题构成了后续版本的回归用例。

8.2 展望

方向	计划			
扫描件结构	引入版面检测（表格/公式区域识别），把扫描件的表格与公式也纳入结构化		公式精度	接入公式识别模型（如 pix2tex）替换当前启发式重建
增量索引	支持按页/按章节增量更新，避免整篇重索引			
评测体系	构建带金标答案的问答评测集，量化检索召回与核验准确率			
长任务治理	把解析与整篇翻译改为后台任务队列，提供进度轮询、断点续跑与取消			
多语言	扩展嵌入与翻译到日/德/法等语种			
协作与导出	支持团队共享知识库、导出带锚点的阅读笔记与综述草稿			